DO NOW — RECALL

Records, lists and tuples

Name: _____ Class: _____ Date: _____

1. What does names[2] access in a 1D array?

2. In a C# 2D array organised as rows and columns, which dimension does GetLength(1) measure?

3. Which structure from last lesson has a fixed size once created?

4. Why should a student's name, candidate number and score be stored as separate related fields rather than one primitive value?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

The third element. Array indexes start at 0.

2. Model answer

The size of dimension 1 — the number of columns.

3. Model answer

An array.

4. Model answer

The data item has several related fields that belong together.

By the end, you can…

• describe records, lists and tuples and recognise suitable uses;

• create and access C# lists and tuples;

• choose between an array, record-like item, list and tuple for a problem;

Vocabulary — know these terms

Recall the term — record

Recall the term — field

Recall the term — list

Recall the term — tuple

Recall the term — index

Recall the term — Add()

Recall the term — Remove()

Recall the term — Count

NEW LEARNING

Read, annotate and explain

Records and tuples in C#

Both group related values. A record defines a reusable named type; a tuple groups a fixed set of values without defining a separate type.

Array → fixed ordered values
Record → named fields for one item
List → ordered values that can grow/shrink
Tuple → small fixed group of values

Know what each structure is, when it is useful, and how to use it in a program.

Records group fields

A record represents one entity using several related fields. In C#, this positional record creates a named Student type with properties. Those properties use init, so use a with expression to create an updated copy.

Lists can change size

A list is ordered like an array, but its size can change while the program runs.

Tuples group a few values

A tuple groups values without declaring a reusable Student type. It is especially helpful when a method returns several results.

Choose by behaviour

The exam may ask why one structure suits a context better than another.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Build and use a C# record

Declare the record type

```
public record Student(string Name, int CandidateNumber, int Score);
```

Create a student

```
Student pupil = new Student("Amelia", 1037, 72);
```

Access a property

```
Console.WriteLine(pupil.Score); // 72
```

Update using with

```
Student updated = pupil with { Score = 80 };
Console.WriteLine(updated.Score); // 80
```


WE DO — CHECK AND IMPROVE

Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

Guided 1 — CHECK

Fill the same code
CODE:
names.Add("Daisy");
names.Remove("Ben");

Guided 2 — CHECK

Fill the same code
CODE:
var result = (Name: "Amelia", Score: 72);
Console.WriteLine(result.?);
Console.WriteLine(result.?);
CODE:
var result = (Name: "Amelia", Score: 72);
Console.WriteLine(result.Name);
Console.WriteLine(result.Score);

Guided 3 — CHECK

Complete the decision
CODE:
Need ordered items
Need add/remove
Size changes
Best structure = ?
CODE:
Need ordered items
Need add/remove
Size changes
Best structure = List

Guided 4 — CHECK

Complete the fields
CODE:
Book record
Title = ?
Author = ?
ISBN = ?
CODE:
Book record
Title = "Dune"
Author = "Frank Herbert"
ISBN = "978..."

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking.

A C# List<T> can grow and shrink while the program is running.

Circle: Confident / Getting there / Need help

Explain what you can now do independently.

GREEN PEN — Correct and explain

Choose one error. Write the correct method and explain precisely why your first response was wrong.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

Guided 1 — TRY: List operations

Complete code that adds Daisy then removes Ben.

```
names.?("Daisy");
names.?("Ben");
```

Method / working:

Guided 2 — TRY: Tuple access

A tuple stores a student's name and score.

```
var result = (Name: "Amelia", Score: 72);
Console.WriteLine(result.?);
Console.WriteLine(result.?);
```

Method / working:

Guided 3 — TRY: Choose a structure

A playlist must grow and shrink while the program is running.

```
Need ordered items
Need add/remove
Size changes
Best structure = ?
```

Method / working:

Guided 4 — TRY: Declare a C# record

Define a Book record with Title, Author and ISBN fields, then create one book.

```
Book record
Title = ?
Author = ?
ISBN = ?
```

Method / working:

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

Question 1

Create and edit a list
Create a List<string> of four names. Add a fifth, remove one name, then output every remaining item.

Question 2

Group a result
Create a tuple containing a student name and integer score. Output both named values.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

Question 3

Find an item
Create a list of five course names. Ask for a course and report whether the list contains it.

Question 4

Choose and code a record
Write short comments explaining when you would choose an array, list, record-like structure and tuple.